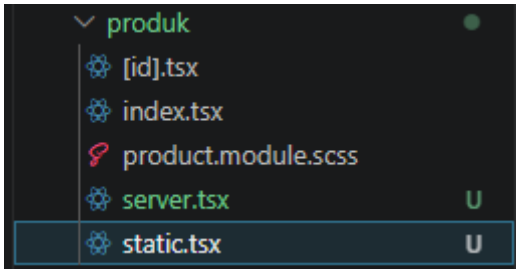
KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
https://jti.polinema.ac.id

Mata Kuliah	:	Pemograman Framework
Program Studi	:	D4 – Teknik Informatika
Semester	:	6

Kelas	:	TI – 3B
NIM	:	2341720169
Nama	:	Dimas Adi Bayu S
Jobsheet ke-	:	10

Laporan Jobsheet

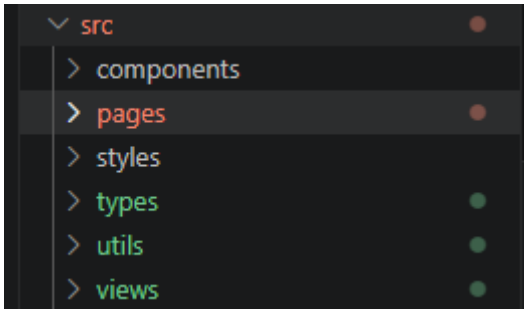
Praktikum 1

Langkah	Jawaban/Deskripsi
1	Buat file baru pada pages/products/static.tsx 
2	Modifikasi file static.tsx

	src > pages > produk > static.tsx > getStaticProps Windsurf: Refactor | Explain | Generate JSDoc | X Windsurf: Refactor | Explain | Generate JSDoc | X

```
import TampilanProduk from "../views/product";
import { ProductType } from "../types/Product.type";

const halamanProdukStatic = (props:{products: ProductType[]}) => {
    const { products } = props;
    return (
        <div>
            <h1>Halaman Produk Static</h1>
            <TampilanProduk products={products} />
        </div>
    )
}

export default halamanProdukStatic;

export async function getStaticProps() {
    const res = await fetch("http://localhost:3000/api/produk");
    // const data = await res.json();
    const response: { data: ProductType[] } = await res.json();
    return {
        props: {
            products: response.data,
        },
    };
}
```

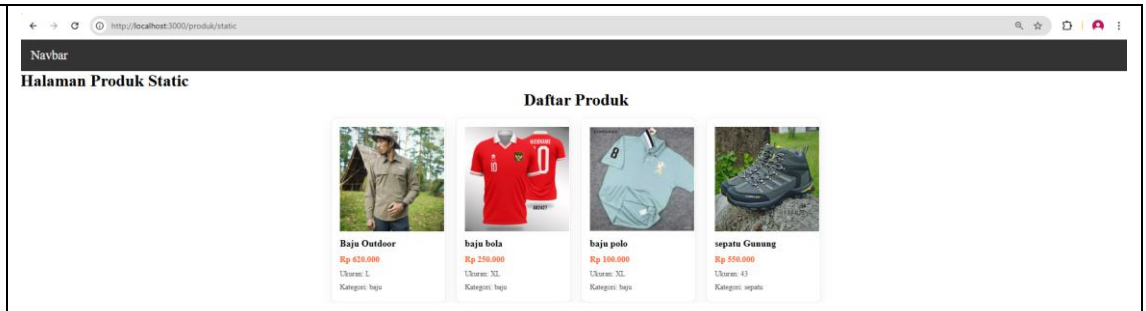
Praktikum Ke-2

Langkah	Jawaban/Deskripsi
1	Pindah beberapa folder diluar pages antara lain - Untuk menghindari error maka folder Views, utils, styles dipindah di luar folder src sehingga susunan folder pada src sebagai berikut 
2	Jalankan: npm run build

	- Jalankan npm run dev dan pastikan ini jalan (jangan distop saat ngebuild), jadi buka dua terminal o Terminal 1 : jalankan aplikasi npm run dev o Terminal 2 : build aplikasi - Hal ini dikarenakan aplikasi berjalan di local http://localhost:3000/api/produk (jika berhasil maka akan muncul pesan route(pages)) PS D:\kuliah\smt6\framework\Tugas\my-app> npm run build ├ o /404 ├ o /about ├ ƒ /api/hello ├ ƒ /api/produk ├ o /auth/login ├ o /auth/register ├ o /blog ├ o /blog/[slug] ├ o /category/[...slug] ├ o /products ├ ƒ /products/server ├ o /produk ├ o /produk/[id] ├ ƒ /produk/server ├ ● /produk/static (869 ms) ├ o /profile ├ o /profile/edit

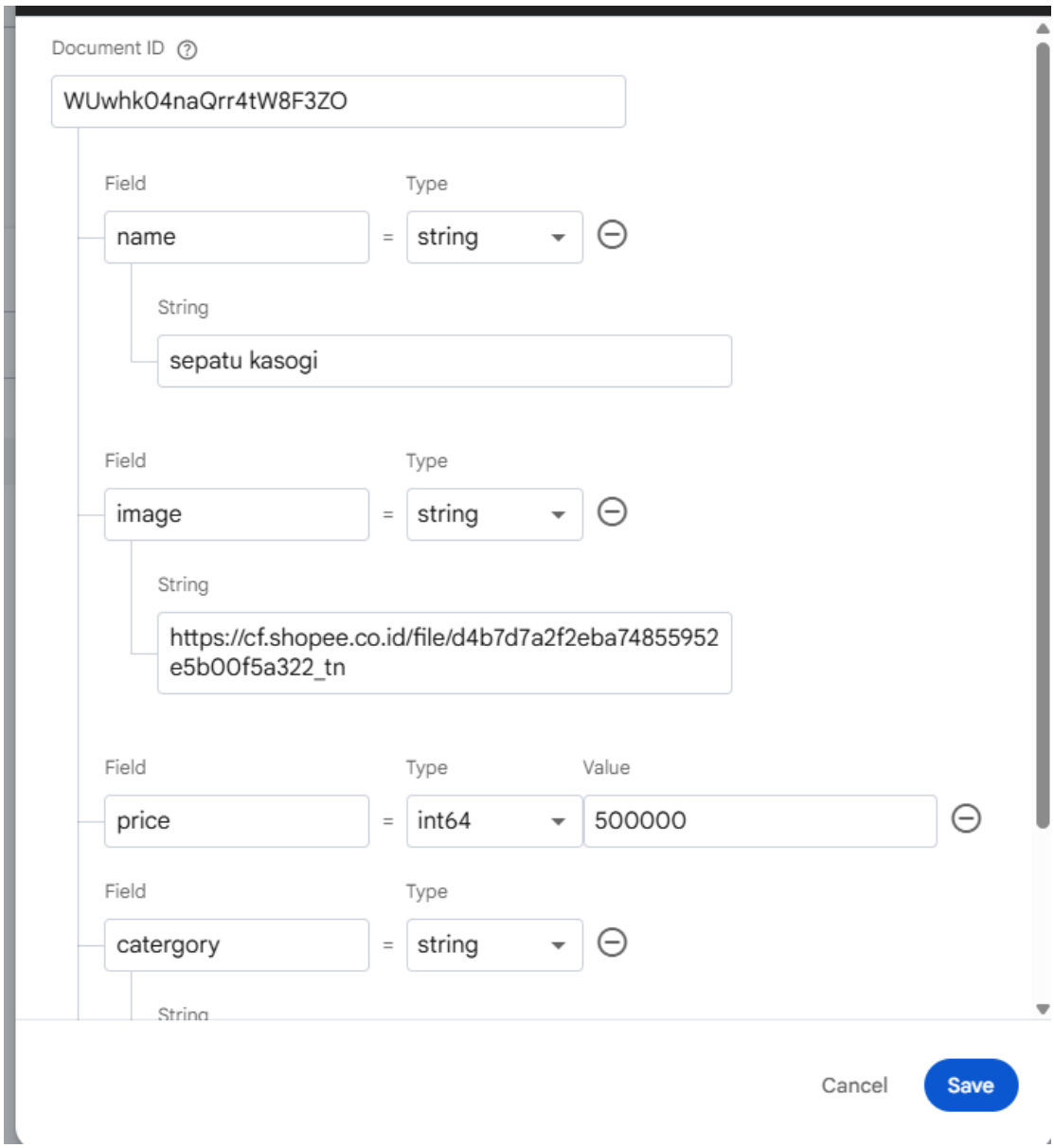
3	Jika berhasil: npm run start
	PS D:\kuliah\smt6\framework\Tugas\my-app> npm run start
	> my-app@0.1.0 start
	> next start
	× Failed to start server
	Error: listen EADDRINUSE: address already in use :::3000
	at <unknown> (Error: listen EADDRINUSE: address already in use :::3000)
	at new Promise (<anonymous>) {
	code: 'EADDRINUSE',
	errno: -4091,
	syscall: 'listen',
	address: '::',
	port: 3000
	}

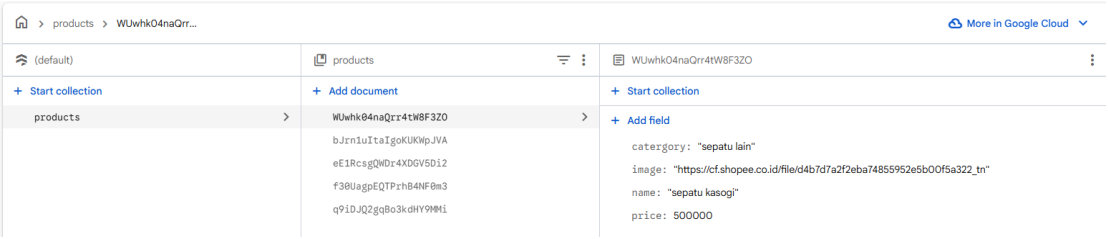
4	Akses: http://localhost:3000/produk/static

Praktikum Ke-3

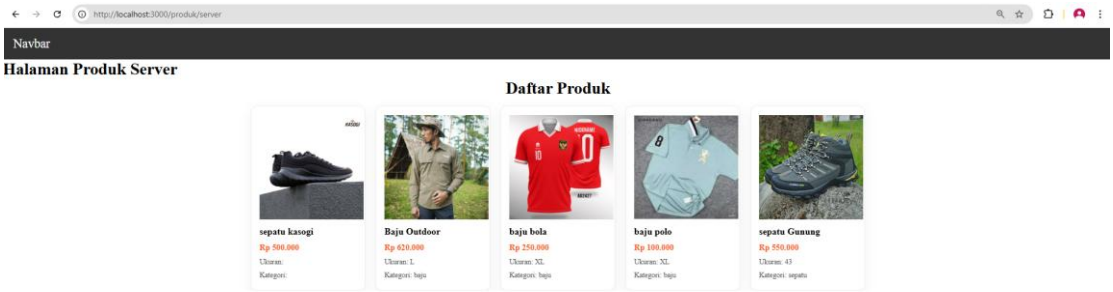
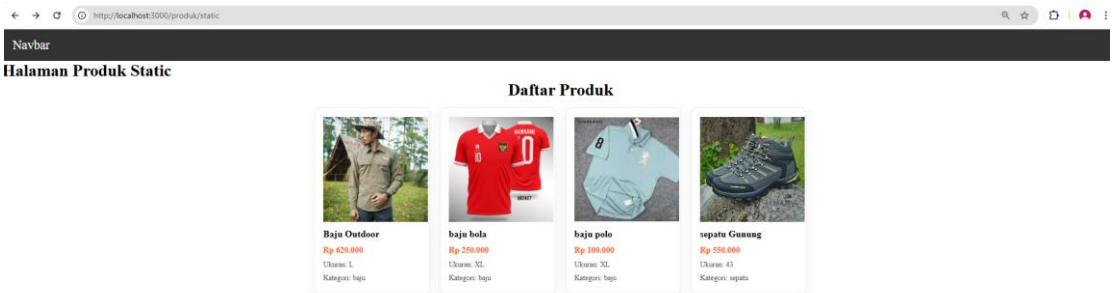
Langkah	Jawaban/Deskripsi
1	Uji 1 – Tambah Data di Database 1. Buka database firebasenya - Tambahkan produk baru di database.

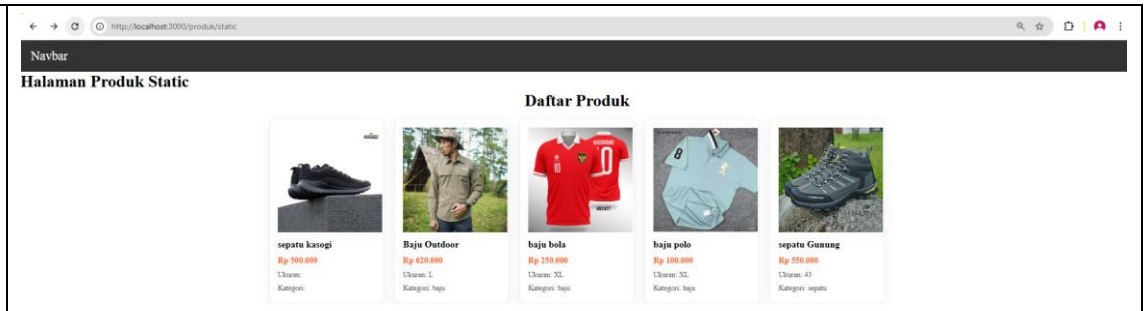
<table>
<tr>
<td></td>
<td>




</td>
</tr>
<tr>
<td>2</td>
<td>Buka halaman:

• /products (CSR) → Data bertambah</td>
</tr>
</table>

3	• /products/server (SSR) → Data bertambah 
4	/products/static (SSG) → Data tidak berubah 
5	Uji 2 – Build Ulang 1. Jalankan kembali: • npm run build o lakukan secara bersamaan dengan npm run dev saat melakukan npm run build • npm run start o npm run dev stop terlebih dahulu setelah itu npm run start 2. Refresh halaman static → Data baru muncul

Tugas 1

Langkah	Jawaban/Deskripsi
1	Buat 3 halaman: o CSR

```
src > pages > csr.tsx > ...
import { useEffect, useState } from 'react';
import TampilanProduk from '../views/product';
import { ProductType } from '../types/Product.type';

Windsurf: Refactor | Explain | Generate JSDoc | X
const CSRPage = () => {
    const [products, setProducts] = useState<ProductType[] | null>(null);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        Windsurf: Refactor | Explain | Generate JSDoc | X
        const fetchProducts = async () => {
            try {
                const res = await fetch('/api/produk');
                const response: { data: ProductType[] } = await res.json();
                setProducts(response.data);
            } catch (error) {
                console.error('Error fetching products:', error);
            } finally {
                setLoading(false);
            }
        };

        fetchProducts();
    }, []);

    if (loading) {
        return <div>Loading...</div>;
    }

    return (
        <div>
            <h1>Halaman CSR (Client-Side Rendering)</h1>
            <p>Data produk di-fetch di sisi client setelah halaman dimuat.</p>
            <TampilanProduk products={products} />
        </div>
    );
};

export default CSRPage;
```

o SSR

src > pages > ssr.tsx > ...

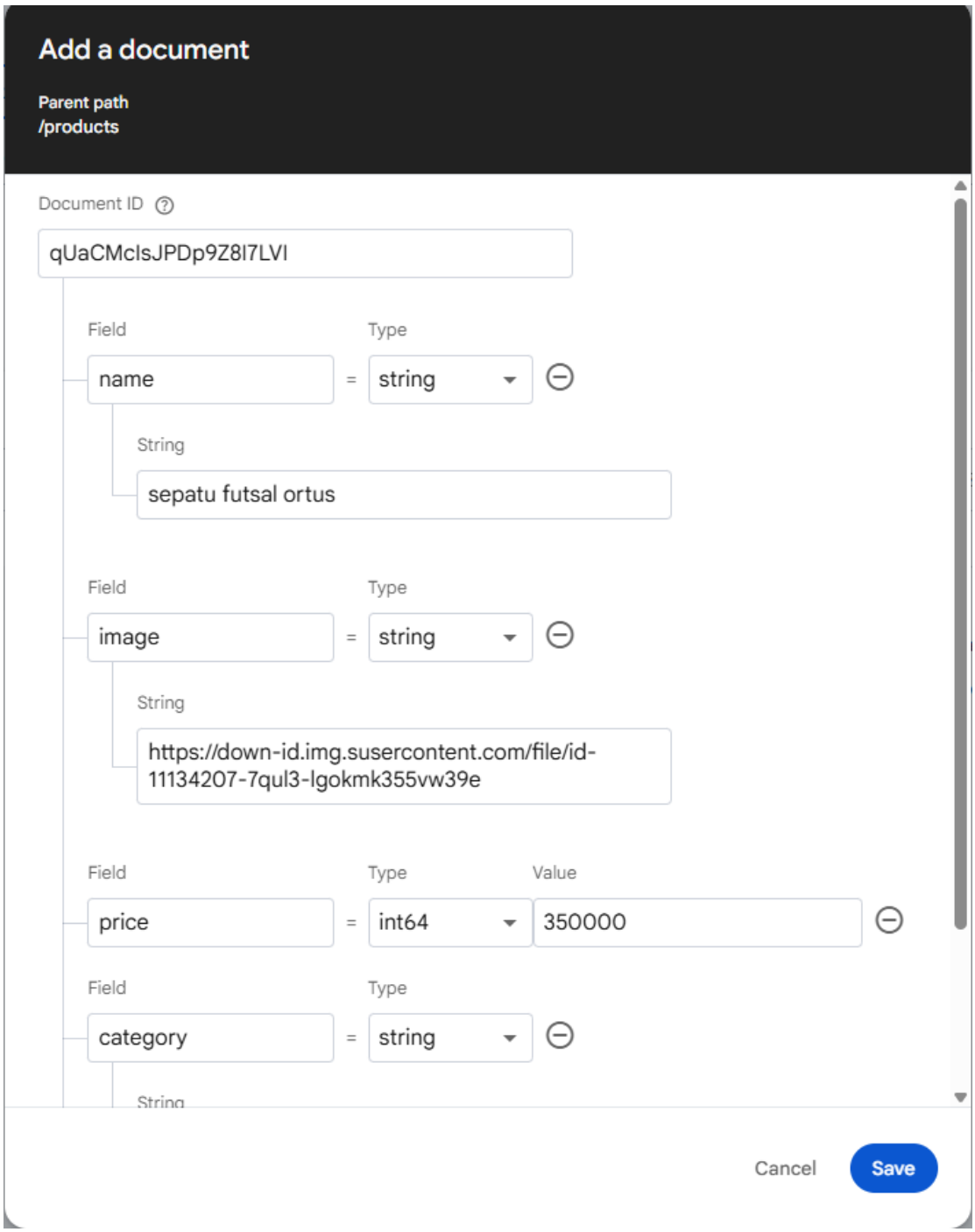
```
import TampilanProduk from '../views/product';
import { ProductType } from '../types/Product.type';

Windsurf: Refactor | Explain
interface SSRPageProps {
    products: ProductType[];
}

Windsurf: Refactor | Explain | Generate JSDoc | X
const SSRPage = ({ products }: SSRPageProps) => {
    return (
        <div>
            <h1>Halaman SSR (Server-Side Rendering)</h1>
            <p>Data produk di-fetch di sisi server pada setiap request.</p>
            <TampilanProduk products={products} />
        </div>
    );
};

export default SSRPage;

Windsurf: Refactor | Explain | Generate JSDoc | X
export async function getServerSideProps() {
    try {
        const res = await fetch('http://127.0.0.1:3000/api/produk');
        const response: { data: ProductType[] } = await res.json();

        return {
            props: {
                products: response.data,
            },
        };
    } catch (error) {
        console.error('Error fetching products:', error);
        return {
            props: {
                products: [],
            },
        };
    }
}
```

o SSG

src > pages > ssg.tsx > ...

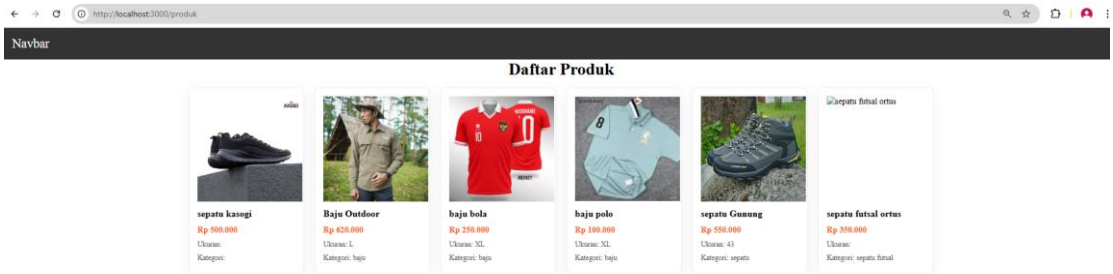
```
import TampilanProduk from '../views/product';
import { ProductType } from '../types/Product.type';

Windsurf: Refactor | Explain
interface SSGPageProps {
    products: ProductType[];
}

Windsurf: Refactor | Explain | Generate JSDoc | X
const SSGPage = ({ products }: SSGPageProps) => {
    return (
        <div>
            <h1>Halaman SSG (Static Site Generation)</h1>
            <p>Data produk di-fetch saat build dan halaman di-generate secara statis.</p>
            <TampilanProduk products={products} />
        </div>
    );
};

export default SSGPage;

Windsurf: Refactor | Explain | Generate JSDoc | X
export async function getStaticProps() {
    try {
        const res = await fetch('http://127.0.0.1:3000/api/produk');
        const response: { data: ProductType[] } = await res.json();

        return {
            props: {
                products: response.data,
            },
            revalidate: 60, // Re-generate setiap 60 detik jika ada request
        };
    } catch (error) {
        console.error('Error fetching products:', error);
        return {
            props: {
                products: [],
            },
        };
    }
}
```

2	Lakukan pengujian: o Tambah data



o Hapus data

o Bandingkan hasil

	
3	Buat laporan analisis minimal 3 halaman. Praktikum ke-10 ini berfokus pada implementasi Static Site Generation (SSG) sebagai salah satu strategi rendering unggulan di Next.js, sekaligus membandingkannya dengan metode yang telah dipelajari sebelumnya, yaitu CSR (Client-Side Rendering) dan SSR (Server-Side Rendering). Analisis ini mencakup aspek teknis pembuatan halaman, manajemen struktur folder, hingga pengujian reaktivitas data terhadap perubahan di database Firebase.1. Implementasi Teknis dan Transformasi Struktur ProyekTahap awal praktikum dilakukan dengan membuat halaman produk statis menggunakan fungsi getStaticProps(). Fungsi ini menjadi kunci dalam SSG karena memerintahkan Next.js untuk mengambil data dari API (/api/produk) pada saat proses build, bukan saat halaman diakses oleh pengguna. Menariknya, untuk menghindari error dependensi selama proses build, dilakukan restrukturisasi folder di mana folder views, utils, dan styles dipindah ke luar folder src. Hal ini menunjukkan bahwa proses build SSG memerlukan lingkungan yang sangat stabil karena Next.js akan melakukan kompilasi menyeluruh terhadap seluruh aset dan data yang diperlukan sebelum aplikasi siap dijalankan.2. Mekanisme Kecepatan dan Efisiensi SSGBerdasarkan hasil pengujian, halaman SSG (/produk/static) terbukti memiliki performa akses yang jauh lebih cepat dibandingkan CSR dan SSR. Hal ini terjadi karena pada mode SSG, browser tidak perlu melakukan komputasi berat atau menunggu respons server yang sedang menyusun HTML. Browser hanya perlu mengunduh file HTML statis yang sudah siap saji. Namun, konsekuensi dari kecepatan ini adalah hilangnya reaktivitas data secara langsung. Pada Uji 1, ketika data produk baru ("sepatu kasogi") ditambahkan ke Firebase, halaman CSR dan SSR langsung menampilkan perubahan tersebut, sementara

	halaman SSG tetap menampilkan data lama. Data baru pada halaman SSG hanya muncul setelah dilakukan build ulang (npm run build) dan menjalankan aplikasi kembali dengan npm run start.3. Analisis Penggunaan pada Kasus Nyata (E-Commerce)Melalui analisis ini, ditemukan bahwa SSG murni memiliki batasan besar jika diterapkan pada platform e-commerce berskala besar. E-commerce membutuhkan data yang bersifat real-time, seperti perubahan stok barang atau fluktuasi harga yang terjadi setiap menit. Jika menggunakan SSG murni, pengguna mungkin akan melihat informasi harga atau stok yang sudah kedaluwarsa karena data tersebut "terkunci" pada saat build terakhir dilakukan. Oleh karena itu, SSG lebih tepat digunakan untuk halaman statis seperti blog, dokumentasi, atau halaman "Tentang Kami" yang jarang berubah. Untuk e-commerce, strategi yang lebih cocok adalah penggunaan Incremental Static Regeneration (ISR), yang memungkinkan regenerasi halaman statis secara berkala di latar belakang tanpa build ulang total, sebagaimana terlihat pada kode tugas yang menyertakan properti revalidate: 60.4. Perbedaan Lingkungan Pengembangan (Dev vs Build)Analisis terakhir menyoroti perbedaan krusial antara Development Mode dan Build Mode. Dalam development mode, Next.js mengutamakan kemudahan pengembang melalui fitur fast refresh, di mana perubahan kode atau data sering kali langsung terlihat untuk mempercepat proses koding. Namun, dalam build mode, fokus utama adalah optimasi performa. Seluruh halaman dikompresi dan dipra-render menjadi file statis agar aplikasi memiliki waktu muat yang minimal saat diakses oleh pengguna akhir di lingkungan produksi.Kesimpulan AnalisisSSG menawarkan performa terbaik dan efisiensi server yang luar biasa karena beban kerja dilakukan "di depan" sebelum aplikasi dirilis. Namun, pengembang harus bijak dalam menentukan kapan menggunakan SSG, SSR, atau CSR tergantung pada seberapa sering data berubah dan seberapa penting aspek SEO bagi halaman tersebut. Integrasi Firebase dengan SSG menuntut pengembang untuk memahami siklus hidup build agar data yang ditampilkan tetap relevan bagi pengguna

Pertanyaan

Langkah	Jawaban/Deskripsi

1	Jawab pertanyaan berikut: 1. Mengapa SSG tidak menampilkan data terbaru? 2. Mengapa SSG lebih cepat? 3. Kapan SSG tidak cocok digunakan? 4. Mengapa e-commerce tidak cocok menggunakan SSG murni? 5. Apa perbedaan build mode dan development mode? Jawab : 1. SSG tidak menampilkan data terbaru karena proses pengambilan data hanya dilakukan sekali pada saat proses build (sebelum aplikasi dijalankan). Jadi, jika ada perubahan data di database setelah aplikasi aktif, tampilan tidak akan berubah sampai aplikasi di-build ulang. 2. SSG lebih cepat karena browser langsung menerima file HTML statis yang sudah jadi. Server tidak perlu lagi mengambil data ke database atau menyusun kode setiap kali ada pengunjung yang datang. 3. SSG tidak cocok digunakan untuk halaman yang memiliki data yang sangat sering berubah (sangat dinamis) atau halaman yang kontennya berbeda-beda untuk setiap pengguna (seperti halaman profil akun). 4. E-commerce membutuhkan data yang real-time, seperti stok barang yang bisa habis kapan saja atau perubahan harga. Jika menggunakan SSG murni, pembeli mungkin melihat barang masih tersedia padahal stoknya sudah habis di database. 5. Development Mode (npm run dev): Digunakan saat menulis kode; fitur seperti Hot Module Replacement aktif sehingga perubahan kode langsung terlihat tanpa merusak alur kerja. Build Mode (npm run build): Digunakan untuk memproduksi versi final aplikasi; semua halaman SSG akan dibuat menjadi file statis agar aplikasi berjalan sangat cepat dan optimal saat diakses pengguna.